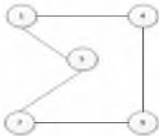


GRAFOS

- EJEMPLO 1: $G=(V, A)$



– $V = \{1, 4, 5, 7, 9\}$.

– $A = \{(1,4), (5,1), (7,9), (7,5), (4,9), (4,1), (1,5), (9,7), (5,7), (9,4)\}$

CONCEPTOS Y DEFINICIONES

- Adyacencia

- Diremos que dos vértices «a» y «b» son adyacentes si existe un arco que une al vértice «a» con el vértice «b».
- En particular en un grafo dirigido diremos que «b» es adyacente a «a» si existe un arco dirigido que sale de «a» y termina en «b»

- Grafos Conexos

- Grafos no dirigidos

- Si hay un camino entre cualquier par de vértices del grafo

- Grafos dirigidos

- Un grafo dirigido es fuertemente conexo si hay un camino entre cualquier par de vértices del grafo.
- Un grafo es débilmente conexo, si no hay camino entre cualquier par de vértices, pero al transformarlo en un grafo no dirigido se tienen un grafo conexo.

- Grafo completo

- Es aquel que tiene un arco para cualquier par de vértices.



CONCEPTOS Y DEFINICIONES

- GRADO

- En un grafo no dirigido el grado de un vértice v , es el numero de aristas que contiene a v .

- En un grafo dirigido se distingue entre grado de entrada y grado de salida.

- Grado de entrada de un vértice v , es el número de arcos que llegan al nodo v

- Grado de salida de un vértice v , es el número de arcos que salen del nodo v



CONCEPTOS Y DEFINICIONES

- CAMINO

–Un camino de longitud n , desde el vértice v_0 al vértice v_n en un grafo es la secuencia de $n + 1$ vértices $v_0, v_1, v_2, \dots, v_n$ tal que la arista (v_i, v_{i+1}) pertenece al conjunto de aristas del grafo.

GRAFOS

- REPRESENTACION COMPUTACIONAL

- Matriz de adyacencia:

- Los vértices se representan por índices (0..n)
 - Los arcos entre los vértices se guardan en una matriz

- Lista de adyacencia:

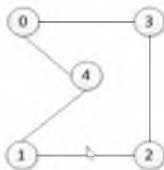
- Los vértices forma una lista simple ordenada (Sin repetidos)
 - Cada vértice tienen una lista para representar los arcos que salen de el.
 - Nota: También podríamos implementar utilizando un mapa donde la llave sería genérica y representaría un vértice y cada valor del mapa para un vértice, sería una lista ordenada genérica de adyacentes al vértice sin repetidos.

GRAFOS

- Matriz de adyacencia:

- Se registra 1 si un vértice es adyacente a otro y 0 si no lo es.

—Ejemplo 1:

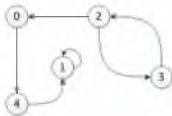


	0	1	2	3	4
0	0	0	0	1	1
1	0	0	1	0	1
2	0	1	0	1	0
3	1	0	1	0	0
4	1	1	0	0	0

—En un grafo no dirigido la matriz es simétrica

GRAFOS

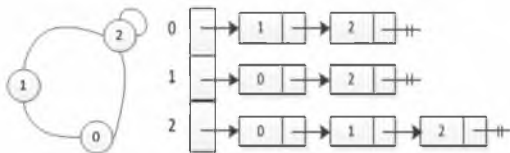
- Matriz de adyacencia (cont.)
 - Ejemplo 2



	0	1	2	3	4
0	0	0	0	0	1
1	0	1	0	0	0
2	1	0	0	1	0
3	0	0	1	0	0
4	0	1	0	0	0

GRAFOS

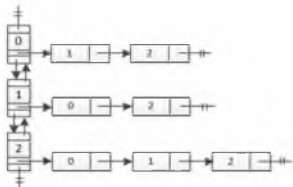
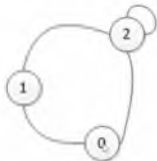
- Lista de Adyacencia
 - La colección de listas puede ser un vector



GRAFOS

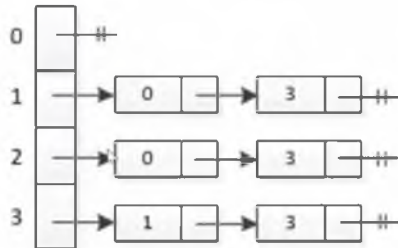
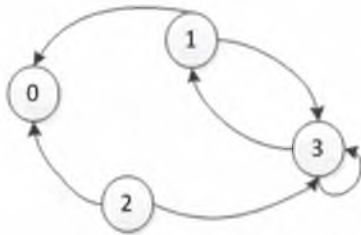
- Lista de adyacencia (Cont.)

- La colección de listas puede ser una lista doblemente enlazada (Multilista)



GRAFOS

- Lista de adyacencia
 - Ejemplo 3



GRAFOS

- La class grafo
 - Normalmente en un programa no se hace uso de una class grafo genérica, si no mas bien el grafo se moldea a la necesidad del programador.
- Operaciones de la class grafo
 - insertarVertice (): inserta un nuevo vértice, correlativo desde el 0.
 - insertarArista(int a, int b): crea una arista entre a y b

GRAFOS

- Operaciones de la class grafo
 - eliminarVertice(int vértice): Elimina el vértice, borrando las aristas que salen o llega a el.
 - eliminarArista(int a, int b): Elimina la arista entre a y b
 - cantidadDeVertices(): Retorna la cantidad de vértices en el grafo.
 - cantidadDeAristas(): Retorna la cantidad de aristas en el grafo.

GRAFOS

- Implementación

```
public class Grafo {  
    private List<List<Integer>> listasDeAdyacencias; //las listas de enteros ordenadas  
    private boolean esDirigido;  
  
    public Grafo() {  
        this(false);  
    }  
  
    public Grafo(boolean esDirigido) {  
        listasDeAdyacencias = new ArrayList<>();  
        this.esDirigido = esDirigido;  
    }  
    .....  
}
```

GRAFOS

- Implementación de la segunda versión de la class grafo

```
public class Grafo<E extends Comparable> {  
    private List<E> listaDeVertices;  
    private List<List<Integer>> listasDeAdyacencias;  
    private boolean esDirigido;  
  
    public Grafo() {  
        this(false);  
    }  
  
    public Grafo(boolean esDirigido) {  
        listasDeAdyacencias = new ArrayList<>();  
        listaDeVertices = new ArrayList<>();  
        this.esDirigido = esDirigido;  
    }  
    .....  
}
```



GRAFOS

- Recorridos en grafos

- Un recorrido (Search) es simplemente la forma en que se visitan los nodos de un grafo, a partir de un nodo dado.
- En grafos, básicamente existen dos formas de recorrerlos:
 - Breadth First Search (BFS), recorrido en anchura o amplitud.
 - Depth First Search (DFS), recorrido en profundidad
- La implementación de las formas de recorridos, requiere de un mecanismo de marcado para cuando un vértice o nodo ya se procesa.



GRAFOS

- BFS

- Este método comienza visitando el vértice de partida A, para a continuación visitar los adyacentes que no estuvieran ya visitados. Así sucesivamente con los adyacentes. Este método utiliza una cola como estructura auxiliar en la que se mantienen los vértices que se vayan a procesar posteriormente.

GRAFOS

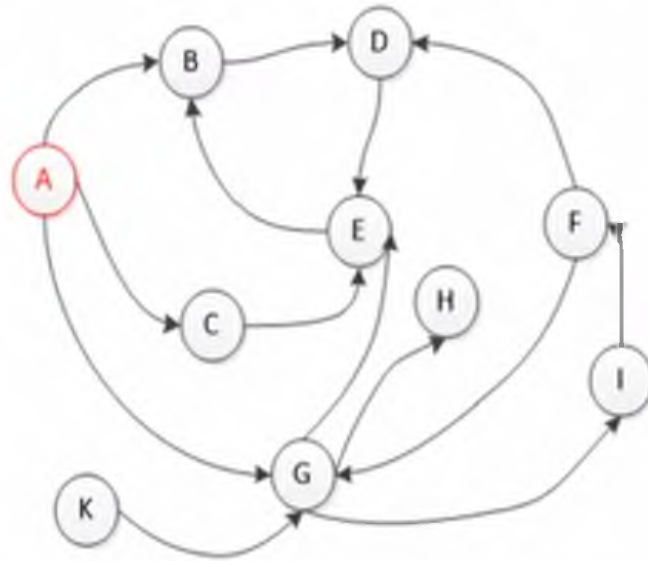
- BFS – Pasos

- 1.Iniciar el proceso por el vértice de partida A.
- 2.Meter en la cola el vértice de partida y marcarle como procesado.
- 3.Repetir los pasos 4 y 5 hasta que la cola esté vacía.
- 4.Retirar el vértice frente (W) de la cola, visitar W.
- 5.Meter en la cola todos los vértices adyacentes a W que no estén procesados y marcarlos como procesados .

- Requiere que todos los nodos inicialmente estén marcados como no procesados



GRAFOS – BFS (Ejemplo)



COLA	
Frente	Atras
A	
B C G	
C G D	
G D E	
D E H I	
E H I	
H I	
I	
F	
Vacía	

RECORRIDO

BFS:

BFS: A

BFS: A, B

BFS: A, B, C

BFS: A, B, C, G

BFS: A, B, C, G, D

BFS: A, B, C, G, D, E

BFS: A, B, C, G, D, E, H

BFS: A, B, C, G, D, E, H, I

BFS: A, B, C, G, D, E, H, I, F

MARCADOS

A

A, B, C, G

A, B, C, G, D

A, B, C, G, D, E

A, B, C, G, D, E, H, I

A, B, C, G, D, E, H, I

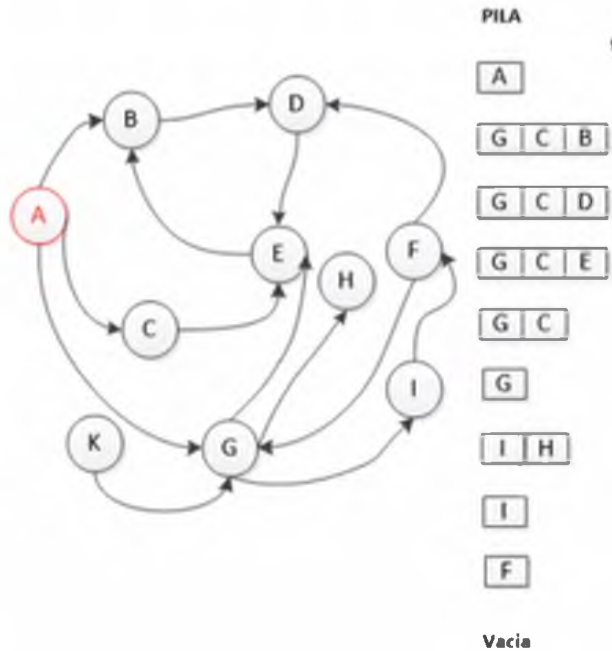
A, B, C, G, D, E, H, I

A, B, C, G, D, E, H, I

A, B, C, G, D, E, H, I, F

A, B, C, G, D, E, H, I, F

GRAFOS – DFS (Ejemplo)



RECORRIDO

DFS:

DFS: A

DFS: A, B

DFS: A, B, D

DFS: A, B, D, E

DFS: A, B, D, E, C

DFS: A, B, D, E, C, G

DFS: A, B, D, E, C, G, H

DFS: A, B, D, E, C, G, H, I

DFS: A, B, D, E, C, G, H, I, F

MARCADOS

A

A, G, C, B

A, G, C, B, D

A, G, C, B, D, E

A, G, C, B, D, E

A, G, C, B, D, E

A, G, C, B, D, E, H, I

A, G, C, B, D, E, H, I

A, G, C, B, D, E, H, I, F

A, G, C, B, D, E, H, I, F

GRAFOS

- La class grafo puede ayudarse para implementar los recorridos de un arreglo booleano que tenga tantas ocurrencias como vértices tenga el grafo. Este arreglo tendrá false, cuando el vértice asociado a la misma posición aun no sea procesado y tendrá true cuando el vértice ya se procese.

Matriz de caminos

- Matriz de caminos.

- Es la matriz que almacena 1 para un par de vértice a y b , si existe un camino de cualquier longitud que inicia en a y finaliza en b .
- Si A es la matriz de adyacencia de un grafo G , se la llamara matriz de caminos de longitud 1.
- Si multiplicamos $A \times A$, se obtendrá la matriz de caminos de longitud 2 (A_2)
- Si multiplicamos $A \times A_2$ se obtendrá la matriz de caminos de longitud 3 (A_3)
- En general si multiplicamos $A \times A_{n-1}$ se obtendrá la matriz de caminos de longitud n (A_n)
- Si realizamos la suma lógica de todas las matrices, tendremos la matriz P de caminos del grafo G
- OTRO MODO DE OBTENERLO ES EL ALGORITMO DE WARSHALL.

- CIERRE TRANSITIVO:

- El cierre transitivo de un grafo G es otro grafo G' que consta de los mismos vértices que G y que tiene como matriz de adyacencia la matriz de caminos P del grafo G . Según esta definición, un grafo es fuertemente conexo si y sólo si su cierre transitivo es un grafo completo.